SPOTIFY: DISTRIBUTED PARADIGMS AND ALGORITHMS

Distributed Computing | University of Hyderabad

Student	K. Amulya	**Reg. No.**	23MCCE22
App schoosen	Spotify	**Developed app**	SimpleStream (Spotify-like app in Java)
Versions	Spotify	**Developed version**	SimpleStream v1.0,

1. Introduction and Scope

This report identifies the **distributed paradigms** and the **distributed algorithms** used in the design of Spotify, and in my own app, SimpleStream, a Spotify-like music application in pure Java sockets. It explains each one briefly and shows where it sits in the architecture and which goal it serves: **scalability, availability or fault tolerance. Paradigms**.

Spotify facts come from sources written by Spotify's engineers wherever possible: the Spotify Engineering blog, a research paper by two Spotify engineers, Spotify for Developers documentation, Spotify Support and Spotify's own white paper.

2. Versions Studied

Subject	Version / stage	What is covered (source)
Original Spotify	P2P-assisted desktop (2008–2014)	Servers plus peer-to-peer streaming on desktop; mobile server-only; P2P removed in April 2014
Original Spotify	Own data centres, then Google Cloud (2015–2018)	About 2,000 services, 20,000 daily pipeline runs and over 100 PB of data moved; all traffic on Google Cloud by May 2017 Events rebuilt on Pub/Sub and Dataflow
Original Spotify	Containers and Kubernetes (2014 onward)	Helios from 2014, then Kubernetes; thousands of microservices [4]. Public Web API, Player API, Web Playback SDK
SimpleStream	Mini Spotify(Developed)	Website, terminal client and 3 socket nodes (ports 5001–5003) with round-robin balancing, write-all replication, Lamport clock and persistence . The project has no version number; v1.0 is the label used here.

3. Original Spotify: Distributed Paradigms

The table lists the paradigms visible in Spotify's published design, with the evidence and the goal each one serves.

Paradigm	Evidence in Spotify's sources	Goal served
Client–server	Each client keeps one long-lived, multiplexed TCP connection to a Spotify server; smartphone clients stream only from servers	Simple, controllable core; availability of music through servers
Peer-to-peer (hybrid)	Desktop clients also fetched audio from other clients through an unstructured overlay with a tracker; 35.8% of played data came from peers. Removed in 2014	Scalability: offloads servers (only 8.8% from servers)
Microservices	Thousands of microservices in production, one above 10 million ruests/s [4]; each owned by an autonomous squad	Scalability and independent deployment
Containers and orchestration	Docker with Helios from 2014, then Kubernetes; new service creation fell from up to an hour to seconds or minutes; CPU utilisation up two to three times	Scalability, efficient resource use
Replication and geo-distribution	Cassandra clusters replicated between two North American and two European data centres so users are served from a nearby site	Availability and low latency
Horizontal partitioned storage	Cassandra chosen for horizontal scaling; doubling the cluster roughly halved median and 99th-percentile latency	Scalability

Paradigm	Evidence in Spotify's sources	Goal served
Publish–subscribe / event-driven	Cloud Pub/Sub carries all events, one topic per event type, about 1,000 event types [12].	Decoupling, availability of the pipeline
Stream and batch processing	Dataflow streaming jobs replace hourly batch export; older design used Kafka and Hadoop MapReduce	Scalability, lower latency
Cloud computing	Move from four own data-centre regions to Google Cloud, 2015–2018.	Elastic capacity, lower operations load
Remote control (Connect)	One device is the controller and another plays; only the active device plays; state shared across all Connect devices	Transparent multi-device use
API-based (service-oriented)	Web API and Player API as HTTP endpoints with OAuth tokens and scopes	Open access, security

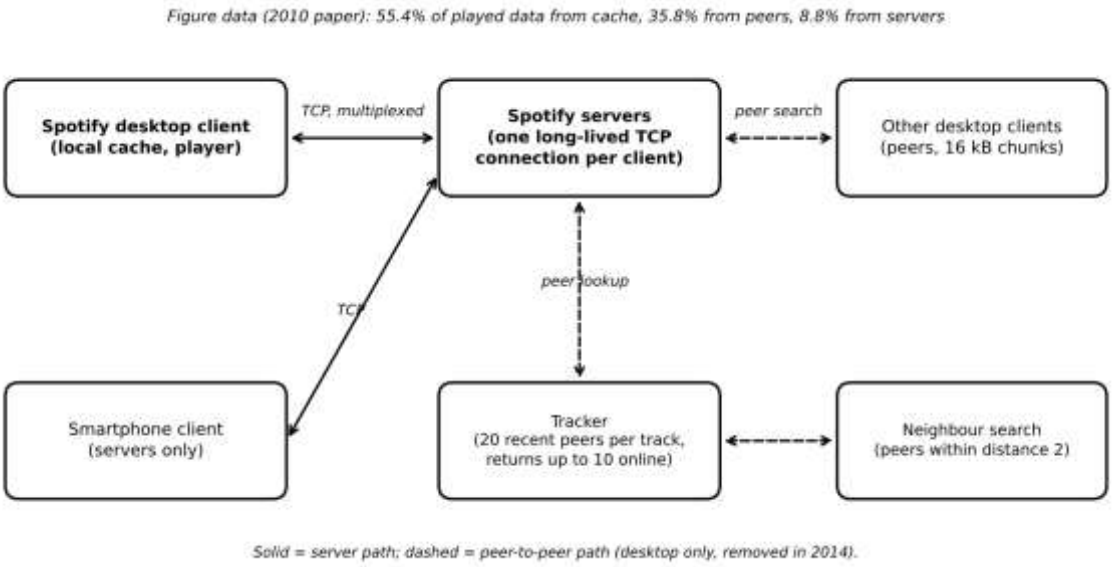


Figure 1. Spotify's client–server and peer-to-peer streaming paths in the P2P era (from [1]).

4. Developed App: Distributed Paradigms

SimpleStream demonstrates the same families of ideas at small scale. Every hop is a raw `java.net.Socket`; the three nodes run on localhost on ports 5001–5003 (Figure 2).

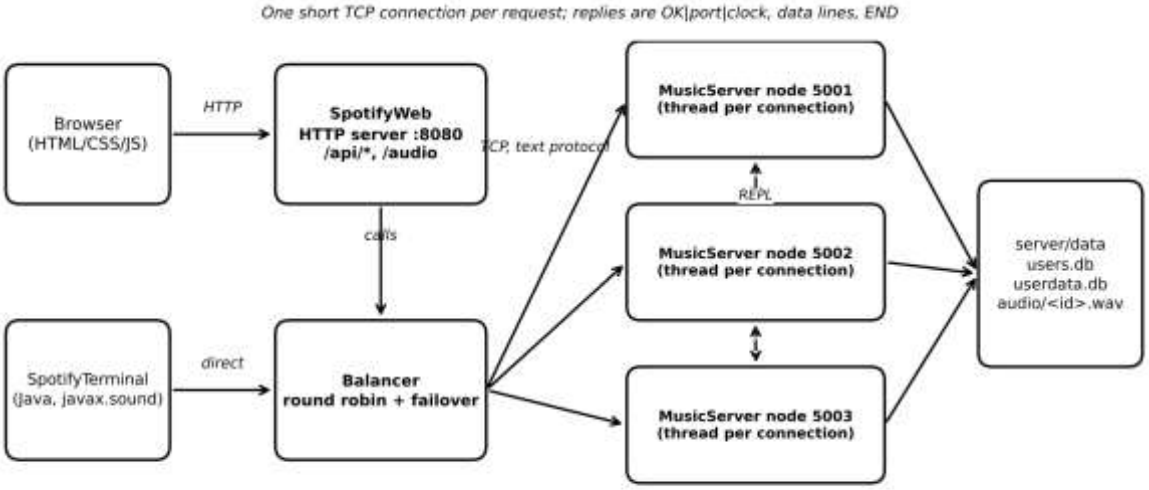


Figure 2. Architecture of SimpleStream v1.0.

Paradigm	Where it appears in SimpleStream	Goal served
Client–server	Website (via `SpotifyWeb`) and `SpotifyTerminal` are clients; `MusicServer` nodes are servers [16].	Separation of UI and data
Three-tier gateway design	Browser → `SpotifyWeb` (HTTP gateway) → `Balancer` → `MusicServer`.	Hides the cluster from the browser
Replication (write-all)	A write is applied on the receiving node and forwarded as `REPL` to the other two (`MusicServer.replicate`).	Availability: any node can answer reads
Load balancing	`Balancer.send()` picks the next node for each request.	Scalability of reads
Fault tolerance and failover	If a node does not answer, `Balancer` tries the next; "Crash last node" demonstrates it.	Fault tolerance
Logical time	Lamport clock in `MusicServer.handle()`; shown as `OK|port|clock`.	Ordering of events across nodes
Multithreading and concurrency	One thread per connection; `synchronized` protects data and file writes.	Many simultaneous users, safe updates
Stateless requests	NEXT/PREV carry the current song and playlist, so any node can answer.	Any node can serve any request
Persistence	`users.db` and `userdata.db` saved after every write and reloaded at start-up.	Durability across restarts
Location and access transparency	The user sees one service; the node that answered is only shown in the sidebar.	Transparency

5. Original Spotify: Distributed Algorithms and Techniques

The sources name specific methods, which are grouped below by the goal they serve. These are the algorithms and techniques Spotify's engineers describe, not an inventory of its code.

Goal	Algorithm / technique	How Spotify uses it
Scalability	Tracker-based peer discovery	Tracker maps each track to the 20 most recent peers and returns up to 10 online ones.
Scalability	Neighbour query flooding (limited)	Search queries are forwarded to neighbours, reaching peers within distance two; the last 50 query IDs are remembered to drop duplicates.
Scalability	Chunked transfer and peer ranking	16 kB chunks requested in order; peers sorted by expected download time; upload to at most 4 peers at a time.
Scalability	Bin-packing scheduling	Kubernetes packs containers onto machines, improving CPU use two to three times.
Latency	LRU caching	Encrypted local cache, LRU eviction; 55.4% of data came from cache.
Latency	Prefetching	Next track searched at 30 s left and its start fetched from the server at 10 s; without it median start rose from 265 ms to 390 ms.
Latency	Message prioritisation	Messages multiplexed on one TCP connection are prioritised; browsing beats bulk transfer.
Availability	Cross-site replication	Cassandra replicates between EU and NA data centres; users are served from the nearest.
Consistency	Tunable consistency	Consistency levels tuned to favour latency over strict consistency; writes about 10× faster than reads.
Fault tolerance	Server fallback	While searching for peers the client also requests audio from the server; so peers are not the only source of audio (interpreted).

Goal	Algorithm / technique	How Spotify uses it
Fault tolerance	Idempotent output	Dataflow writes one file per pane using the pane's unique ID, so retries after failure do not duplicate data.
Fault tolerance	Windows, watermarks, late-data rules	Hour-long event-time windows; early and late triggers; events later than one day are dropped; completeness marker per bucket.
Data loading	Bulk SSTable streaming	Data built in HDFS as SSTables and streamed into Cassandra (hdfs2cass).

5.1 Peer discovery, chunk scheduling and playback (Spotify client)

The client first looks in its local cache. On a miss it asks the server over its open connection and, on desktop, looks for peers (Figure 3). For a random-access play (about 39%) it asks the server for about 15 seconds of audio at once; for a play in sequence (about 61%) it starts looking for peers 30 seconds before the end of the current track. The paper notes that Spotify does not use a distributed hash table; it combines a tracker with limited neighbour search to distance two [1].

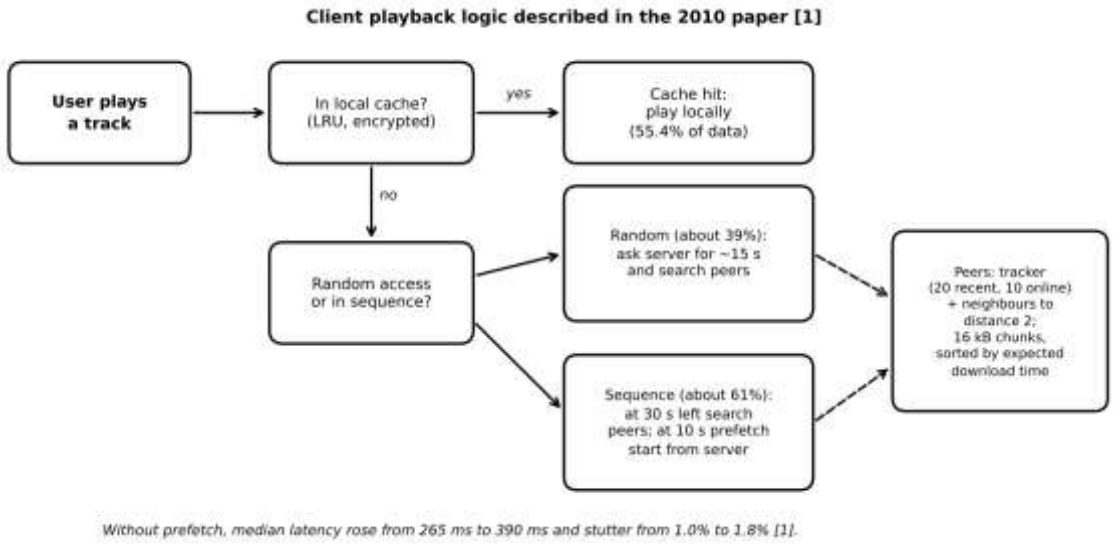


Figure 3. Playback decision logic in the Spotify desktop client (from [1]).

5.2 Event delivery: from a single point of failure to managed pub/sub

The older pipeline moved events through Kafka and Groupers to one central Hadoop cluster and relied on end-of-file markers from every sender. Hadoop was a single point of failure, and a marker never arrived if a machine died, so a stalled sender could hold up a whole hourly bucket [11]. The new design gives each event type its own Pub/Sub topic and uses Dataflow with event-time windows and per-pane IDs (Figure 4) [12].

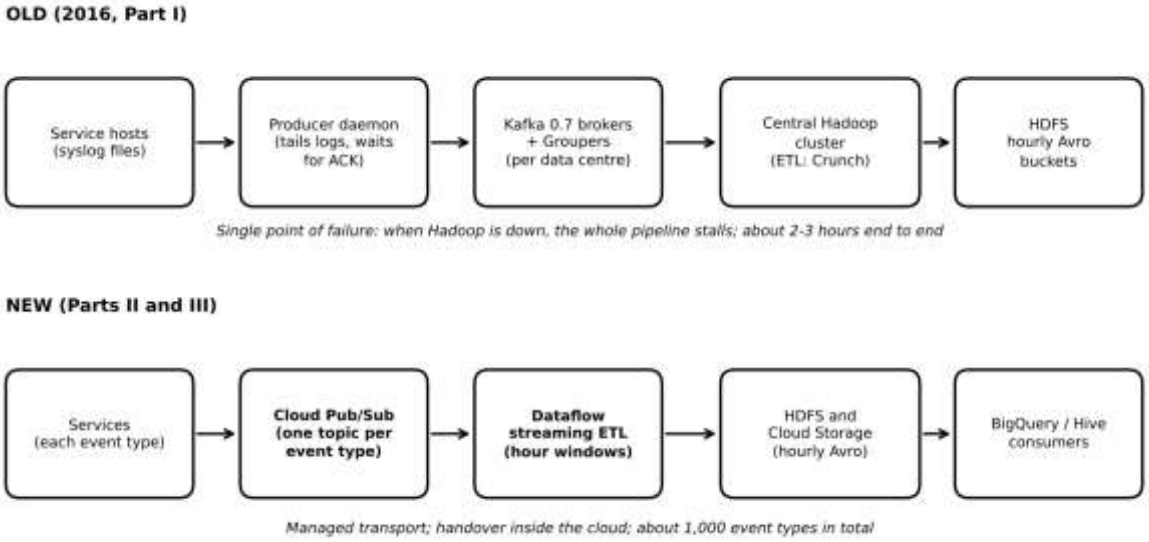


Figure 4. Old and new event-delivery designs at Spotify (from [11][12]).

5.3 Replicated storage with tunable consistency

Spotify chose Cassandra because it needed horizontal scaling, cross-site replication, low latency rather than strict consistency, bulk and streaming loads and flexible schemas. Two clusters hold entity metadata and user profiles; the team tuned consistency levels to favour latency, split a table into static and TTL-only tables to help compaction, and monitored SSTables per read to diagnose latency [13].

6. Developed App: Distributed Algorithms

Goal	Algorithm	How it works in SimpleStream	Where
Scalability	Round-robin load balancing	`start = counter++ % 3`; the first node tried is 5001, 5002, 5003, 5001, ...	`Balancer.send()`
Availability, fault tolerance	Failover by retry on next node	Up to 3 nodes tried; 800 ms connect and 4 s read timeouts; error only if all fail.	`Balancer.send()`, `ask()`
Availability	Write-all replication	A write is applied locally, then `REPL<TAB>clock<TAB>command` is sent to every other node (500 ms connect, 1 s read); dead nodes are skipped.	`MusicServer.replicate()`
Ordering	Lamport logical clock	A write increments the clock; a replica sets `clock = max(own, received) + 1`.	`MusicServer.handle()`
Consistency of local state	Mutual exclusion by lock	`synchronized (this)` around apply and save; `synchronized (MusicServer.class)` around file I/O.	`MusicServer`
Durability	Atomic save (temp file + move)	Write `userdata.db.tmp`, then `Files.move` with REPLACE_EXISTING after every write.	`saveData()`
Availability	Stateless request handling	NEXT/PREV carry the current song and playlist; no per-session state on a node.	`apply()` case NEXT/PREV
Failure detection	Liveness probe (PING)	`PING` returns `OK`; the sidebar shows each node as up or down.	`Balancer.isUp()`
Concurrency	Thread per connection	Each accepted socket is served by a new thread.	`MusicServer` accept loop

6.1 Round-robin balancing with failover

`Balancer` is used by both the website and the terminal. A shared counter, incremented under a lock, picks the first node; if that node fails to connect or reply, the loop moves to the next one (Figure 5). Because the choice is made on the client side, there is no separate load-balancer machine to fail. A node that is down is simply skipped on every request, so the service keeps running while at least one node is up.

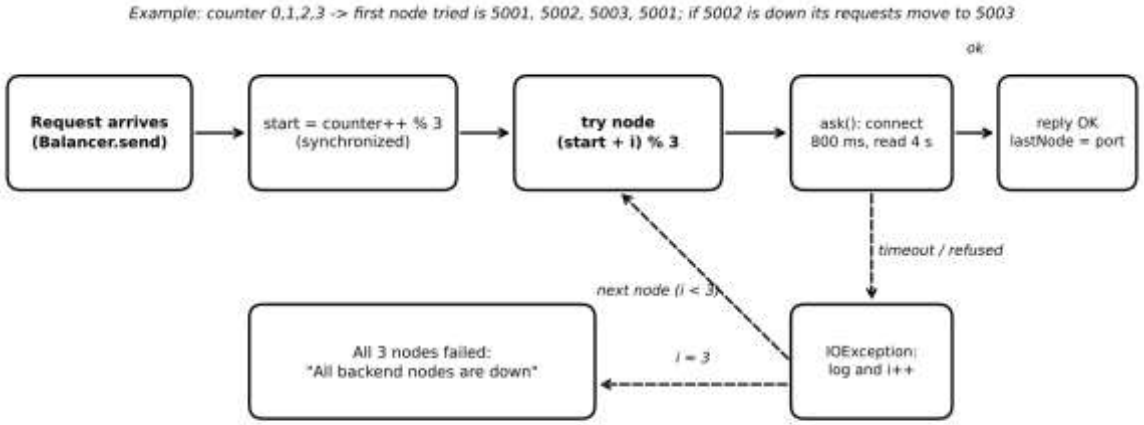


Figure 5. Round-robin selection and failover in `Balancer.send()`.

6.2 Write-all replication

A write command (`SIGNUP`, `LIKE`, `PLCREATE`, `PLDELETE`, `PLADD`, `PLREMOVE`, `PLAY`) is executed on the node that received it, which saves to disk and then forwards the same command to the other nodes before replying to the client (Figure 6). Reads can then be answered by any node. Replication is synchronous and best effort: a node that is down is skipped and is not brought up to date when it returns, so the whole backend must be restarted [16].

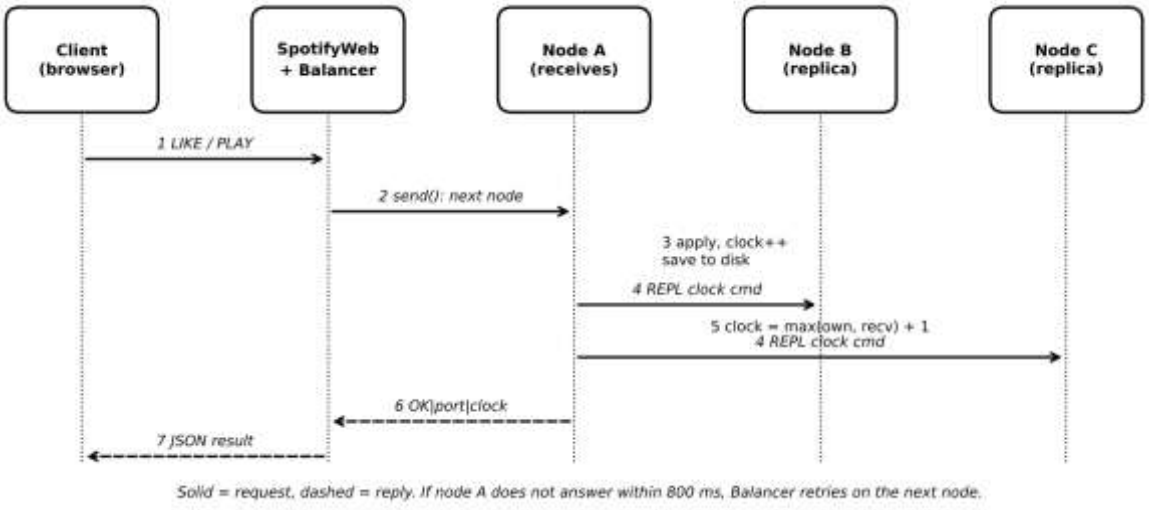


Figure 6. Life of a write: receive, apply, replicate, reply with the clock.

6.3 Lamport logical clock

Each node keeps a counter. A client write increments it; a replicated write sets it to one more than the larger of the node's own value and the received value. Figure 7 shows two writes on different nodes: the clock numbers increase along every message, so the order of related events can be read from the numbers shown in the terminal and the backend log. A Lamport clock gives an ordering consistent with causality, but it does not by itself resolve conflicting concurrent writes.

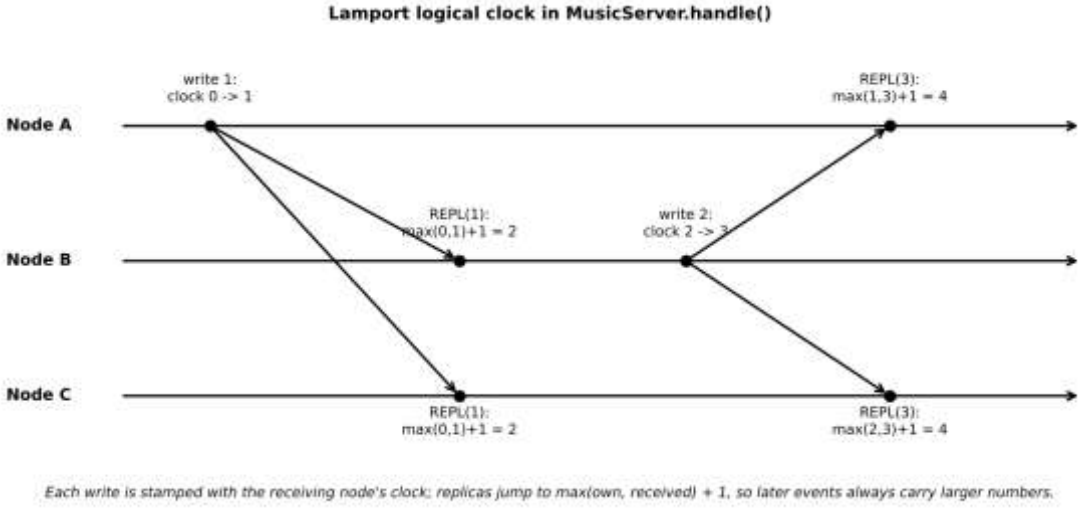


Figure 7. Lamport clock updates across three nodes.

6.4 Concurrency control, atomic persistence and stateless design

- **Locking:** the lock is held only while local data is changed and saved, not while replicating, so a slow replica does not block other requests on the same node.
- **Atomic save:** the new file is written completely before it replaces the old one, so a crash during a save cannot leave a half-written `userdata.db`.
- **Stateless commands:** because NEXT and PREV carry their own context, the balancer can send them to any node, and no sticky sessions are needed.

7. Goals Achieved: Scalability, Availability and Fault Tolerance

Goal	Original Spotify (paradigm / algorithm)	Developed app (paradigm / algorithm)
Scalability	Microservices on Kubernetes with bin-packing Cassandra horizontal scaling peer offload and caching so only 8.8% of data came from servers; Pub/Sub and Dataflow for events	Round-robin spreads requests over three nodes (read scalability). Writes are applied on every node, so adding nodes does not increase write capacity.
Availability	Replication between EU and NA data centres; multi-region cloud local cache serving most data	Write-all replication means any node can answer reads; failover to the next node in `Balancer`.
Fault tolerance	Server fallback beside P2; removal of the Hadoop single point of failure ; idempotent Dataflow output	Failover and dead-node skipping; atomic file save; “Crash last node” demonstration. A restarted node is not re-synced.
Consistency and ordering	Tunable consistency, latency favoured event-time windows and watermarks	Lamport clock orders writes; replication is synchronous but best effort.
Latency	LRU cache, prefetching, prioritised multiplexed messages	HTTP Range requests for seeking; no caching or prefetching.
Transparency	One service across regions; Connect treats all devices as one	One service view; node shown only in the sidebar.

8. Summary of my findings

- **Same paradigms, different scale.** Both systems are client–server with replication, load balancing and persistence; Spotify adds peer-to-peer (until 2014), microservices, event-driven pipelines and cloud infrastructure.
- **Spotify’s algorithms target latency and scale.** The tracker, limited neighbour search, chunk scheduling, LRU caching and prefetching are all about starting playback quickly; the published numbers (265 ms median start, 8.8% of data from servers) show the effect [1].

- **SimpleStream's algorithms target correctness and visibility.** Round robin, failover, write-all replication and the Lamport clock are small enough to read in a few methods, and each can be demonstrated from the interface.
- **Failure lessons match.** Spotify's event-delivery post concludes that one stalled component must not stop everything; SimpleStream's equivalent weakness is that a crashed node cannot rejoin.
- **Consistency choices are similar in spirit.** Spotify favours latency over strict consistency; SimpleStream applies writes locally first and skips dead replicas, also favouring availability.

9. Conclusion

Spotify's published design combines client–server and (formerly) peer-to-peer streaming, microservices on Kubernetes, replicated Cassandra storage, a Pub/Sub event pipeline and cloud hosting, supported by algorithms for peer discovery, caching, prefetching, tunable consistency and idempotent processing [1][3][4][12][13]. SimpleStream v1.0 implements the core of the same toolbox in miniature: client–server, load balancing with failover, write-all replication, a Lamport clock, locking, atomic persistence and stateless requests. What it leaves out (node recovery, quorum writes, caching, real database) is exactly what Spotify's engineers describe as the work needed for real scale.

10. References (accessed October 2026)

- Backend infrastructure at Spotify — Spotify Engineering (2013)
- Achieving Fault-Tolerance with Intelligent Daemons — Spotify Engineering (2013)
- Spotify's Event Delivery – The Road to the Cloud, Part I — Spotify Engineering (2016)
- Spotify's Event Delivery – The Road to the Cloud, Part II — Spotify Engineering (2016)
- Helios (Spotify's earlier open-source container orchestrator) — Spotify GitHub
- Apache Cassandra documentation — gossip, consistent hashing, tunable consistency
- Apache Kafka documentation — replication and partition leaders
- Kubernetes documentation — etcd and the Raft consensus algorithm

- **23MCCE22 (K.AMULYA)**